

Optimización de Sistemas de Inferencia: Análisis Técnico de TurboQuant para la Compresión de la Caché KV en Arquitecturas Transformer

La evolución de las arquitecturas de aprendizaje profundo hacia modelos de lenguaje de gran escala (LLM) con capacidades de razonamiento sofisticadas ha desplazado el centro de gravedad de la optimización desde el rendimiento puramente aritmético hacia la gestión eficiente de la jerarquía de memoria. En la inferencia de modelos basados en la arquitectura Transformer, el crecimiento lineal de la caché de llaves y valores (Key-Value Cache o KV Cache) en relación con la longitud de la secuencia de entrada representa el principal obstáculo para el escalado del contexto y la eficiencia del rendimiento por usuario. El sistema TurboQuant, desarrollado por Google Research y presentado formalmente para el ICLR 2026, emerge como una metodología de cuantización vectorial online (en tiempo de inferencia) diseñada para abordar estas limitaciones mediante un enfoque geométrico que busca alcanzar la tasa de distorsión óptima bajo restricciones de latencia críticas.¹

Introducción Arquitectónica: El Paradigma de la Inferencia en la Era del Contexto Extenso

En el diseño de sistemas de inferencia para LLMs modernos, la gestión de la caché KV es fundamental para evitar el re-cálculo redundante de estados ocultos en cada paso de generación de tokens. Tradicionalmente, los modelos han operado en precisiones de punto flotante de 16 bits (FP16 o BF16), lo que implica que para modelos con ventanas de contexto masivas (como 128K o incluso 1M de tokens), la huella de memoria de la caché KV puede superar significativamente el tamaño de los pesos del modelo en sí mismos.³ Esta problemática se agudiza en arquitecturas densas y en escenarios de alto rendimiento donde el ancho de banda de la memoria del acelerador se convierte en el cuello de botella dominante, un fenómeno conocido como "memory-bound inference".

TurboQuant se posiciona no solo como una técnica de reducción de datos, sino como un marco de ingeniería de sistemas que reestructura la forma en que los vectores de atención son representados y procesados. A diferencia de las técnicas de cuantización de pesos (como GPTQ o AWQ) que se realizan offline y de forma estática, TurboQuant opera de manera dinámica durante el proceso de autoregresión. La arquitectura de TurboQuant se fundamenta en la premisa de que los vectores en altas dimensiones poseen propiedades geométricas explotables que permiten una compresión mucho más agresiva que los métodos escalares convencionales.¹

El sistema integra dos algoritmos complementarios: PolarQuant, que gestiona la reducción dimensional inicial y la representación geométrica, y la corrección residual mediante la transformada de Johnson-Lindenstrauss cuantizada (QJL), que garantiza que el estimador del producto interno permanezca insesgado.² Esta combinación permite que TurboQuant mantenga una fidelidad de atención que, según las pruebas reportadas, es indistinguible de la precisión completa incluso con presupuestos de apenas 3.5 bits por canal.¹

Atributo Técnico	Cuantización Escalar Tradicional	TurboQuant (Online VQ)
Momento de aplicación	Offline (Pesos) / Inferencia (KV)	Online (Inferencia)
Granularidad	Bloques / Por canal	Vectorial (Alta dimensión)
Presupuesto de bits típico	4 bits / 8 bits	2.5 bits - 4 bits
Necesidad de calibración	Alta (requiere datos)	Nula (Data-oblivious)
Soporte de Producto Interno	MSE-orientado (Sesgado)	Insesgado (vía QJL)

Geometría y Distribución: El Rol de la Rotación Aleatoria de Haar

El núcleo de la eficiencia de TurboQuant reside en su etapa de preprocesamiento geométrico. Antes de cualquier operación de discretización, el sistema aplica una rotación ortogonal aleatoria basada en la medida de Haar a los vectores de entrada.¹ Es imperativo, desde una perspectiva de ingeniería de sistemas, no equiparar erróneamente esta operación con un proceso de "whitening" o blanqueamiento clásico. Mientras que el blanqueamiento implica una normalización de la matriz de covarianza para que sea la identidad, alterando las magnitudes relativas para descorrelacionar los datos, la rotación de Haar es una transformación ortogonal estricta. Esta rotación preserva la norma L_2 de los vectores y simplemente reorienta la

energía de manera uniforme a través de todas las dimensiones del espacio.¹

Análisis de la Concentración de Medida

El efecto de esta rotación es la democratización de la varianza. En los datos naturales, ciertas dimensiones pueden contener la mayor parte de la información (esparsidad o dispersión irregular), lo que dificulta la cuantización uniforme. Tras la rotación ortogonal aleatoria, las coordenadas resultantes del vector transformado se comportan de manera predecible. Los

autores de TurboQuant demuestran que, para un vector unitario de dimensión d , cada coordenada individual sigue una distribución Beta concentrada, específicamente una

$$Beta\left(\frac{1}{2}, \frac{d-1}{2}\right).$$

Esta propiedad es fundamental porque permite diseñar cuantizadores de Lloyd-Max óptimos que son independientes del conjunto de datos específico ("data-oblivious"). Sin embargo, la

investigación también señala que a medida que la dimensión d aumenta, esta distribución converge rápidamente hacia una distribución Gaussiana o Normal debido al teorema central del límite (CLT) aplicado a esferas de alta dimensión y a los fenómenos de concentración de medida.¹ En términos prácticos de implementación, los sistemas pueden modelar estas

coordenadas como variables Gaussianas $\mathcal{N}(0, 1/d)$ para simplificar la creación de tablas de cuantización sin pérdida apreciable de fidelidad.¹

La relevancia de este enfoque geométrico radica en la mitigación de los valores atípicos (outliers). En el espacio original, un valor atípico en una dimensión específica puede forzar un rango de cuantización amplio, degradando la precisión de los valores comunes. Tras la rotación, el impacto de ese valor atípico se diluye entre todas las dimensiones, permitiendo que el cuantizador opere en un régimen de error mucho más estable.⁴

Análisis Profundo del Pipeline: PolarQuant y la Integración de QJL

El flujo de procesamiento de TurboQuant se divide en dos etapas críticas que separan la optimización del error cuadrático medio (MSE) de la preservación de la estructura del producto interno, necesaria para el mecanismo de atención de Softmax.¹

Etapas 1: PolarQuant y el Manejo de Magnitudes

La primera etapa, denominada PolarQuant, se encarga de la compresión principal. El nombre deriva de la transformación de coordenadas cartesianas a una representación de tipo polar (magnitud y ángulos).² Técnicamente, PolarQuant agrupa pares de coordenadas de los vectores ya rotados y utiliza la mayoría del presupuesto de bits disponible (por ejemplo, si el objetivo es 4 bits, se asignan 3 bits aquí) para codificar la dirección y magnitud del vector.⁸

El uso de cuantizadores de Lloyd-Max sobre la distribución Beta inducida permite que PolarQuant alcance una distorsión de MSE que se aproxima al límite inferior de Shannon.¹ Un beneficio colateral de este método es la eliminación de la necesidad de almacenar constantes de normalización por bloque, un requerimiento común en métodos como INT8 o INT4 escalar. En estos métodos tradicionales, el almacenamiento de las escalas y los ceros (offsets) genera un gasto de memoria adicional que puede representar hasta el 10-15% de la caché KV; PolarQuant evita este "impuesto" mediante su enfoque geométrico global.⁶

Etapa 2: Corrección Residual QJL (Quantized Johnson-Lindenstrauss)

Una de las observaciones más perspicaces de la investigación es que los cuantizadores optimizados para minimizar el MSE (como Lloyd-Max) producen estimadores sesgados para el producto interno.¹ Dado que la atención en un Transformer se calcula como

$$Attention(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{d}}\right)V, \text{ cualquier sesgo en el producto interno } QK^T \text{ puede acumularse y degradar la calidad de la salida.}^4$$

Para resolver esto, TurboQuant introduce una segunda etapa que opera sobre el residuo de la primera etapa. Si x es el vector original y $\hat{x}$ es la reconstrucción tras PolarQuant, el residuo es $\epsilon = x - \hat{x}$.⁸ El sistema aplica una transformada de Johnson-Lindenstrauss cuantizada (QJL) a este residuo, utilizando solo 1 bit adicional por dimensión. Este bit captura simplemente el signo de la proyección aleatoria del residuo. El resultado final es un estimador insesgado que preserva las distancias y similitudes con una fidelidad matemática superior, permitiendo que configuraciones de 3 o 3.5 bits se comporten como sistemas de mayor precisión en términos de recuperación de información.¹

Impacto Cuantitativo en la Caché KV: Justificación y Derivación de Métricas

Para evaluar el impacto de TurboQuant en la ingeniería de sistemas, es necesario cuantificar la huella de memoria en modelos de producción. No basta con citar cifras agregadas; se requiere un desglose basado en las especificaciones arquitectónicas.

Justificación de la Métrica para Llama 3 70B

Tomemos como referencia el modelo Llama 3 70B, ampliamente utilizado en benchmarks de la industria. Su arquitectura se define por los siguientes parámetros técnicos ¹¹:

- Capas (L): 80.
- Cabezas de Atención (h): 64.
- Cabezas de Key/Value (h_{kv}): 8 (debido al uso de Grouped-Query Attention o GQA).

- **Dimensión de la cabeza (d_k):** 128 (calculado como $d_{model}/h = 8192/64$).
- **Precisión:** FP16 (2 bytes por parámetro).

La fórmula para calcular el tamaño de la caché KV por token para un modelo dado es:

$$Size_{per_token} = 2 \times L \times h_{kv} \times d_k \times Precision_{bytes}$$

Para Llama 3 70B en FP16:

$$Size = 2 \times 80 \times 8 \times 128 \times 2 = 327,680 \text{ bytes (aprox. 320 KB per token)}$$

13

En un escenario de servicio con **128 usuarios** concurrentes y una ventana de contexto de **32,768 tokens (32K)**:

$$Total = 128 \times 32768 \times 327680 \text{ bytes} \approx 1,374,389,534,720 \text{ bytes} \approx 1.37 \text{ TB}$$

Si el sistema utiliza **FP32** para la acumulación de la caché (común en ciertos frameworks de alta precisión), este valor se duplica a **~2.74 TB**.³ Este volumen de datos es inmanejable para un solo nodo de computación convencional sin técnicas de compresión agresivas.

Configuración (L3-70B, 32K, 128 usuarios)	Precisión	Memoria KV (TB)	Factor de Compresión
Baseline (FP32)	32 bits	~2.68 TB	1x
Estándar (FP16)	16 bits	~1.34 TB	2x
TurboQuant (Alta Fidelidad)	4 bits	~0.33 TB	8x
TurboQuant (Optimizado)	3.5 bits	~0.29 TB	9.1x
TurboQuant (Máximo)	2.5 bits	~0.21 TB	12.8x

Límite)			
---------	--	--	--

En las pruebas reportadas, el uso de 3.5 bits se considera el "punto dulce" donde la neutralidad de calidad es absoluta, permitiendo que un clúster que antes estaba saturado con 16 usuarios ahora pueda manejar a los 128 proyectados en la tabla.¹

Benchmarks de Hardware: Del Ecosistema Hopper al Blackwell

La eficacia de TurboQuant no solo reside en la capacidad de almacenamiento, sino en la velocidad de acceso. En sistemas de inferencia, el rendimiento suele estar limitado por el ancho de banda de memoria (memory bandwidth bottleneck).

Diferenciación Blackwell (B200) vs. DGX B200

Es crucial para el ingeniero de sistemas distinguir entre el rendimiento de un componente individual y el de un sistema integrado.

- Una **GPU Blackwell (B200)** individual cuenta con 180 GB de memoria HBM3e y ofrece un ancho de banda de aproximadamente **8 TB/s**.¹⁴
- Un sistema **DGX B200**, que integra ocho de estas GPUs mediante una red de interconexión NVLink de quinta generación y NVSwitch, alcanza un ancho de banda de memoria agregado de **64 TB/s** y una capacidad de memoria total de 1.4 TB.¹⁵

TurboQuant aprovecha este ancho de banda masivo. Al reducir el tamaño de los vectores de la caché KV en un factor de 4x a 6x, se reduce proporcionalmente el volumen de datos que debe viajar desde la memoria HBM hasta los Tensor Cores durante la fase de atención. En una arquitectura H100, se han reportado aceleraciones de hasta **8x** en el cálculo de los logits de atención específicamente, debido a que la menor huella de memoria permite una mejor utilización de las jerarquías de caché L1/L2 de la GPU.²

Edge AI y Memoria Unificada

En dispositivos móviles y estaciones de trabajo locales (como Apple Silicon), el paradigma cambia a la **memoria unificada** o **RAM del sistema**. Aquí, la GPU y la CPU comparten el mismo pool de memoria física.⁵ En un MacBook M4 con 16 GB de memoria unificada, un modelo de 35B parámetros cuantizado en sus pesos (Q4_K_M) ocupa unos 20 GB, lo que lo hace imposible de ejecutar sin TurboQuant.¹⁷ Al aplicar TurboQuant a la caché KV, el espacio requerido para el contexto se reduce drásticamente, permitiendo que modelos que antes requerían 64 GB de RAM funcionen con fluidez en dispositivos con 32 GB o menos, manteniendo latencias de generación aceptables superiores a 15-20 tokens/s.¹⁷

Críticas Técnicas y Análisis Comparativo: TurboQuant frente a RaBitQ

A pesar de los resultados prometedores, la metodología de TurboQuant ha sido objeto de escrutinio técnico, particularmente en su comparación con RaBitQ (SIGMOD 2024). La comunidad de investigación ha señalado discrepancias significativas que deben ser analizadas con rigor.

Cotas de Error en las Colas y Variabilidad

La crítica fundamental se centra en las garantías teóricas. RaBitQ ha demostrado ser asintóticamente óptimo para la estimación del producto interno, ofreciendo cotas de error en las colas (tail bounds) que escalan de manera extremadamente eficiente: el bit-width necesario para una probabilidad de error δ escala como $\log \log(1/\delta)$.²⁰

Por el contrario, las garantías teóricas de TurboQuant se limitan principalmente al error cuadrático medio (MSE). Al intentar derivar cotas de probabilidad para escenarios extremos (colas de la distribución) utilizando la desigualdad de Chebyshev, el requerimiento de bits para TurboQuant escala como $\log(1/\delta)$. Esto es exponencialmente más lento que RaBitQ, lo que apunta a que TurboQuant podría presentar una **variabilidad de error mayor** en escenarios extremos donde la precisión de un solo valor atípico es crítica para la atención.²⁰

Métrica Teórica	RaBitQ	TurboQuant
Escalamiento de bits ($1/\delta$)	$\log \log(1/\delta)$ (Óptimo)	$\log(1/\delta)$ (Subóptimo)
Estimador de Producto Interno	Insesgado (vía re-normalización)	Insesgado (vía QJL residual)
Independencia de Coordenadas	No requerida explícitamente	Asumida tras rotación

Controversias de Benchmarking y Reproducibilidad

Existen informes persistentes sobre la asimetría en las comparaciones experimentales del

paper de TurboQuant. Investigaciones independientes sugieren que el rendimiento de RaBitQ fue evaluado sobre una implementación en Python en una CPU de un solo núcleo, mientras que TurboQuant utilizó kernels de GPU (A100) altamente optimizados.²⁰ En pruebas simétricas utilizando hardware idéntico, la ventaja de velocidad de TurboQuant se reduce significativamente, y en ciertos conjuntos de datos de búsqueda vectorial (como GloVe o embeddings de OpenAI), RaBitQ mantiene una precisión de recuperación (recall) superior o equivalente con menor complejidad de cómputo.⁷

Evaluación de la Pérdida de Precisión (Perplexity)

En cuanto a la degradación de la calidad del modelo, los resultados sugieren que TurboQuant es extremadamente resiliente en tareas de contexto largo. En el benchmark **LongBench**, las configuraciones de 3.5 bits muestran una perplejidad y puntuaciones de precisión que se mantienen dentro del margen de error estadístico respecto al baseline de FP16.¹

Específicamente, en pruebas con Llama 3 y Mistral, los resultados apuntan a que la perplejidad aumenta menos del 0.5% en la mayoría de las tareas de resumen y extracción de datos, incluso cuando la ventana de contexto se extiende hasta los 128K tokens.⁹ Sin embargo, en las pruebas reportadas para modelos de escala pequeña (menos de 8B parámetros), se ha observado una mayor sensibilidad a la cuantización, donde el ruido introducido por la rotación de Haar puede no ser compensado completamente por la corrección QJL, sugiriendo que la eficacia de TurboQuant escala favorablemente con el tamaño del modelo.⁴

Impacto en el Ecosistema y Mercado Financiero: Perspectivas Actuales

La presentación de TurboQuant en marzo de 2026 provocó una reacción inmediata en los mercados financieros y en los esfuerzos de integración de software.

Integración en vLLM y Frameworks de Código Abierto

El estado real de la integración en **vLLM** (issue #38171) muestra un progreso acelerado. Los desarrolladores han propuesto una extensión del CacheDType para soportar el formato "turboquant", junto con la implementación de kernels de Triton para realizar la rotación y el bit-packing de forma fusionada.²⁵ Los datos preliminares indican una mejora del **21% en el throughput** para batch sizes medianos en GPUs H100.²⁵

En el ecosistema Apple Silicon, existen implementaciones experimentales como turboquant_plus (un fork de llama.cpp) y prototipos en MLX. Aunque los ahorros de memoria son tangibles, la falta de kernels de Metal optimizados para la rotación de Haar a gran escala limita actualmente las ganancias de velocidad de ejecución en comparación con las soluciones de NVIDIA.¹⁸

Análisis de Mercado: Micron y SK Hynix

El impacto financiero del anuncio de TurboQuant ha sido objeto de intensa especulación por parte de los analistas. Tras la publicación del blog de Google Research, se registraron caídas en las acciones de los principales fabricantes de memoria:

- **Micron Technology:** Caída de aproximadamente el 10% el 31 de marzo de 2026.²⁶
- **SK Hynix:** Descenso del 6.2% en sesiones similares.²⁷
- **Samsung Electronics:** Caída cercana al 5%.²⁸

Analistas de firmas como Wells Fargo y Citi han interpretado que este avance tecnológico podría reducir la demanda futura de memoria física (HBM/DRAM) al permitir que los centros de datos operen con infraestructuras más delgadas. No obstante, expertos de Bank of America y Morgan Stanley sostienen una visión contraria, sugiriendo que la mayor eficiencia en el uso de memoria fomentará un despliegue masivo de modelos aún más grandes, lo que en última instancia incrementará el consumo total de bits (la Paradoja de Jevons).²⁶ En ausencia de datos financieros directos que vinculen el anuncio de TurboQuant con cancelaciones de pedidos reales, estas fluctuaciones de mercado deben considerarse como **especulación de analistas** basada en el sentimiento tecnológico a corto plazo.²⁶

Conclusiones Técnicas y Riesgos de Implementación

TurboQuant representa una pieza de ingeniería de sistemas refinada que aborda el cuello de botella más crítico de la inferencia moderna de LLM. Al combinar una rotación de Haar que democratiza la energía de los vectores con una corrección residual QJL que garantiza la integridad del producto interno, el sistema logra una compresión efectiva de 6x sin comprometer la precisión.²

Sin embargo, los riesgos de implementación no son despreciables. La dependencia de la distribución Beta/Gaussiana tras la rotación implica que el sistema es "data-oblivious", lo cual es una ventaja operativa, pero también significa que carece de la adaptabilidad de los métodos basados en datos (como AWQ) para manejar casos de borde arquitectónicos específicos. Además, la controversia sobre las cotas de error en comparación con RaBitQ sugiere que en aplicaciones de altísima sensibilidad (como diagnósticos médicos o cálculos financieros complejos), la variabilidad de TurboQuant en las colas de error podría ser un factor de riesgo. Por último, la ausencia de una implementación oficial de producción por parte de Google a fecha de abril de 2026 obliga a los arquitectos de sistemas a depender de implementaciones de la comunidad que aún requieren una validación de seguridad y estabilidad a gran escala.³

Bibliografía

Fuentes Primarias (Papers)

- Zandieh, A., et al. "TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate." ICLR 2026.¹
- Gao, J., et al. "Revisiting RaBitQ and TurboQuant: A Symmetric Comparison of Methods,

Theory, and Experiments." ArXiv Technical Note, 2026. ⁷

- Han, I., et al. "PolarQuant: Post-Training Weight Quantization for LLMs." AISTATS 2026. ³⁰
- Zandieh, A., et al. "Quantized Johnson-Lindenstrauss (QJL)." 2024. ¹

Fuentes Secundarias (Blogs Técnicos)

- Google Research. "TurboQuant: Redefining AI efficiency with extreme compression." March 2026. ²
- NVIDIA. "DGX B200 Technical Specifications and HBM3e Bandwidth." 2026. ¹⁵
- TIKR. "Financial Analysis: Micron and the AI Memory Trade." 2026. ³²
- Towards Data Science. "How Google fixed the KV Cache bottleneck with TurboQuant." 2026. ⁸

Comentarios de la Comunidad

- vLLM Project. "Issue #38171: Integration of TurboQuant in the KV Cache path." GitHub, 2026. ²⁵
- Reddit r/MachineLearning. "Debate on the consistency of TurboQuant benchmarks." 2026. ³³
- Hacker News. "Discussion on the reproducibility of Google Research's results." 2026. ³⁴
- Llama.cpp Community. "Discussion #20969: TurboQuant for Apple Silicon and Edge Devices." GitHub, 2026. ¹⁷

Works cited

1. TURBOQUANT: ONLINE VECTOR QUANTIZATION ... - OpenReview, accessed April 24, 2026, <https://openreview.net/pdf?id=tO3ASKZlok>
2. TurboQuant: Redefining AI efficiency with extreme compression, accessed April 24, 2026, <https://research.google/blog/turboquant-redefining-ai-efficiency-with-extreme-compression/>
3. Google TurboQuant: 2026 LLM Compression Guide | Articles - O-mega.ai, accessed April 24, 2026, <https://o-mega.ai/articles/google-turboquant-the-2026-llm-compression-guide>
4. Google's TurboQuant: The Compression Breakthrough That Could Reshape LLM Infrastructure | by Akshay Kalane | Mar, 2026 | Towards AI, accessed April 24, 2026, <https://pub.towardsai.net/googles-turboquant-the-compression-breakthrough-that-could-reshape-llm-infrastructure-c09d68017567>
5. Google's TurboQuant Makes LLM Inference Optimization the New AI Battleground - remio, accessed April 24, 2026, <https://www.remio.ai/post/google-s-turboquant-makes-llm-inference-optimization-the-new-ai-battleground>
6. Google TurboQuant: the algorithm that reduces LLM memory by 6x without any

- loss - VeoNum, accessed April 24, 2026,
<https://www.veonum.com/en/google-turboquant-compression-llm-memoire/>
7. Revisiting RaBitQ and TurboQuant: A Symmetric Comparison of Methods, Theory, and Experiments - arXiv, accessed April 24, 2026,
<https://arxiv.org/html/2604.19528v1>
 8. KV Cache Is Eating Your VRAM. Here's How Google Fixed It With TurboQuant., accessed April 24, 2026,
<https://towardsdatascience.com/kv-cache-is-eating-your-vram-heres-how-google-fixed-it-with-turboquant/>
 9. TurboQuant: Google's New Algorithm Compresses LLMs 6x With Zero Accuracy Loss | by Earl Cotten | Newsarticulated | Mar, 2026 | Medium, accessed April 24, 2026,
<https://medium.com/newsarticulated/turboquant-googles-new-algorithm-compresses-llms-6x-with-zero-accuracy-loss-c1077e27412a>
 10. Google TurboQuant: 6x LLM Memory Compression Guide - Digital Applied, accessed April 24, 2026,
<https://www.digitalapplied.com/blog/google-turboquant-6x-llm-memory-compression-guide>
 11. Llama-3 Architecture Overview - Emergent Mind, accessed April 24, 2026,
<https://www.emergentmind.com/topics/llama-3-architecture>
 12. LLM Architecture Design Guide | MaxPool, accessed April 24, 2026,
<https://maxpool.dev/llm-design/>
 13. Serving LLaMA 3-70B on TPUs | How To Scale Your Model - GitHub Pages, accessed April 24, 2026, <https://jax-ml.github.io/scaling-book/applied-inference/>
 14. Nvidia B200 GPU: Specs, VRAM, Price, and AI Performance - Runpod, accessed April 24, 2026, <https://www.runpod.io/articles/guides/nvidia-b200>
 15. NVIDIA DGX B200 Datasheet - Megware, accessed April 24, 2026,
https://www.megware.com/fileadmin/user_upload/LandingPage%20NVIDIA/NVIDIA-dgx-b200-datasheet.pdf
 16. DGX B200: The Foundation for Your AI Factory - NVIDIA, accessed April 24, 2026,
<https://www.nvidia.com/en-us/data-center/dgx-b200/>
 17. Running a 35B Model Locally with TurboQuant — What's Actually Possible Right Now, accessed April 24, 2026,
<https://pub.towardsai.net/running-a-35b-model-locally-with-turboquant-whats-actually-possible-right-now-1ac5327430b0>
 18. Google Dropped TurboQuant Two Weeks Ago. The Community Already Made It Usable., accessed April 24, 2026,
<https://dev.to/alanwest/google-dropped-turboquant-two-weeks-ago-the-community-already-made-it-usable-3h0k>
 19. Googles TurboQuant Changes the Economics of Local AI Inference - Medium, accessed April 24, 2026,
<https://medium.com/@michael.hannecke/googles-turboquant-changes-the-economics-of-local-ai-inference-acce5839014d>
 20. Revisiting RaBitQ and TurboQuant: A Symmetric Comparison of ..., accessed April 24, 2026, <https://arxiv.org/pdf/2604.19528>

21. [Literature Review] Revisiting RaBitQ and TurboQuant: A Symmetric Comparison of Methods, Theory, and Experiments - Moonlight | AI Colleague for Research Papers, accessed April 24, 2026,
<https://www.themoonlight.io/review/revisiting-rabitq-and-turboquant-a-symmetric-comparison-of-methods-theory-and-experiments>
22. TurboQuant and RaBitQ: What the Public Story Gets Wrong - DEV Community, accessed April 24, 2026,
<https://dev.to/gaoj0017/turboquant-and-rabitq-what-the-public-story-gets-wrong-1i00>
23. [2604.19528] Revisiting RaBitQ and TurboQuant: A Symmetric Comparison of Methods, Theory, and Experiments - arXiv, accessed April 24, 2026,
<https://arxiv.org/abs/2604.19528>
24. quantumaikr/quant.cpp: LLM inference with 7x longer context. Pure C, zero dependencies. Lossless KV cache compression + single-header library. - GitHub, accessed April 24, 2026, <https://github.com/quantumaikr/quant.cpp>
25. [Feature]: Add TurboQuant Support for KV Cache Quantization · Issue #38171 · vllm-project/vllm - GitHub, accessed April 24, 2026,
<https://github.com/vllm-project/vllm/issues/38171>
26. Micron Stock Plunges After TurboQuant Shock Hits Market - Evrim Ağacı, accessed April 24, 2026,
<https://evrimagaci.org/gpt/micron-stock-plunges-after-turboquant-shock-hits-market-535920>
27. Google's TurboQuant: Why It Changes Nothing for the Memory Trade | Equity Insights, accessed April 24, 2026,
<https://www.lighthouse-canton.com/insights/turboquant-why-it-changes-nothing-for-the-memory-trade-equity-insights>
28. Beyond the TurboQuant-RaBitQ Debate: Why Vector Quantization Matters for AI Infrastructure Costs - Milvus, accessed April 24, 2026,
<https://milvus.io/blog/turboquant-rabitq-vector-database-cost.md>
29. Micron Slides 5% as Google's AI Memory Algorithm Sparks Fresh Fears Across the Semiconductor Sector - 24/7 Wall St., accessed April 24, 2026,
<https://247wallst.com/investing/2026/03/30/micron-slides-5-as-googles-ai-memory-algorithm-sparks-fresh-fears-across-the-semiconductor-sector/>
30. PolarQuant: Optimal Gaussian Weight Quantization via Hadamard Rotation for LLM Compression - arXiv, accessed April 24, 2026,
<https://arxiv.org/html/2603.29078v1>
31. Introduction to NVIDIA DGX B200 Systems, accessed April 24, 2026,
<https://docs.nvidia.com/dgx/dgxb200-user-guide/introduction-to-dgxb200.html>
32. Micron Stock Has Crashed 30% Since Earnings. Here's Where MU Stock Could Go in 2026, accessed April 24, 2026,
<https://www.tikr.com/blog/micron-stock-has-crashed-30-since-earnings-heres-where-mu-stock-could-go-in-2026>
33. [D] thoughts on the controversy about Google's new paper? : r/MachineLearning - Reddit, accessed April 24, 2026,
https://www.reddit.com/r/MachineLearning/comments/1s7m7rn/d_thoughts_on_th

[e_controversy_about_googles_new/](#)

34. BTW, a number of corrections. The TurboQuant paper was submitted to Arxiv back i... | Hacker News, accessed April 24, 2026,
<https://news.ycombinator.com/item?id=47823002>